Sintesi — Un agente AI costruisce una dashboard time-series

Riproduzione end-to-end del demo "Weather Globe" (Tiger Cloud / TimescaleDB + Claude Code + MCP)

---

1. Obiettivo

Riprodurre, sulla propria macchina Windows, il demo illustrato nel materiale "Daily Dose of Data Science": un agente AI (Claude Code) che, partendo da un solo prompt in linguaggio naturale, **provisiona infrastruttura cloud reale, progetta un database, genera i dati e costruisce un'applicazione web 3D funzionante**.

Lo scopo non è il meteo, né la grafica: è ottenere una demo dal vivo da usare come **materiale didattico** per un pubblico che vuole capire *cosa fa davvero l'AI generativa*. Formato previsto: demo condotta dal docente, con i partecipanti che osservano e analizzano, e possibilità per loro di riprodurla in seguito.

Messaggio centrale della demo: l'AI non si limita a scrivere testo — *agisce*. Il salto concettuale è da "chatbot che completa testo" ad "agente che pianifica azioni, le esegue con strumenti reali, osserva i risultati e si corregge".

---

2. Architettura complessiva

Tre strati, collegati tra loro:

a) L'agente — Claude Code (v2.1.123), modello **Opus 4.7** con effort **xhigh**, in esecuzione nel terminale dentro la cartella di progetto. È il "cervello" che pianifica ed esegue.

b) Il ponte (MCP) — Il server MCP di Tiger, incorporato nella Tiger CLI e registrato dentro Claude Code. È ciò che permette all'agente di **toccare risorse reali**: senza, resterebbe solo testo. Espone tool come `service_create`, `db_execute_query`, `search_docs`.

c) Il backend dati — Un servizio **TimescaleDB su Tiger Cloud** (TimescaleDB è un'estensione di PostgreSQL specializzata per serie temporali). Qui vivono le tabelle, la hypertable e gli aggregati continui.

d) Il frontend — Un'app **Next.js + Three.js + Recharts** generata in locale, che legge dal database via API route.

Flusso dei dati a runtime:

```
[Utente muove lo slider time-travel]
        ↓
[Frontend Next.js chiama una API route, es. /api/snapshot?ts=...]
        ↓
[La route interroga TimescaleDB su Tiger Cloud]
        ↓
[La hypertable risponde leggendo solo il "chunk" temporale richiesto → veloce]
        ↓
[Il globo 3D aggiorna marker, colori e terminatore giorno/notte]
```

È qui che TimescaleDB dimostra il suo valore: ogni movimento dello slider è una query temporale, e la partizione automatica per tempo la mantiene rapida.

---

3. Prerequisiti verificati

Prima di iniziare, ambiente confermato a posto:

- **Account Tiger Cloud** — progetto "New Project", piano Performance, trial 29 giorni attivo, utente Antonio Piemontese (`apiemonte56@gmail.com`).
- **Node.js** — `v22.20.0` (LTS attuale), con **npm** `10.2.3`. Necessario sia per Claude Code sia per il frontend Next.js.
- **Claude Code** — `2.1.123`, già installato e funzionante.
- Ambiente conda `(base)` disattivato per lavorare con strumenti Node-based con prompt pulito.

---

4. I passi eseguiti

Fase A — Installazione e collegamento (i 4 passi di setup)

1. Installazione Tiger CLI (da PowerShell, *non* dal Prompt dei comandi):

```
irm https://cli.tigerdata.com/install.ps1 | iex
```

Esito: *Tiger CLI installed successfully*, **versione 0.20.4**, binario per Windows x86_64 in `…\AppData\Local\Programs\Tiger`, checksum verificato, aggiunto al PATH. (Nota: su `cmd` `irm` / `iex` non esistono — è un comando PowerShell; ed è stato necessario riaprire la finestra per ricaricare il PATH.)

2. Autenticazione:

```
tiger auth login
```

Esito: si è aperto il browser sulla Console, autorizzazione confermata ("You're all set up to use Tiger CLI!"), login riuscito sul **progetto** `pb9zqxrnzp`.

3. Verifica:

```
tiger service list
```

Esito: *No services found* — stato corretto e atteso: nessun servizio ancora, sarà l'agente a crearlo.

4. Collegamento dell'agente via MCP:

```
tiger mcp install claude-code
```

Esito: *Successfully installed Tiger MCP server configuration for claude-code*, configurazione scritta in `C:\Users\Utente\.claude.json`, server disponibile in Claude Code come `tiger`.

Fase B — Avvio dell'agente

Cartella di lavoro e avvio:

```
mkdir $HOME\Desktop\weather-demo
cd $HOME\Desktop\weather-demo
claude
```

- Cartella creata: `C:\Users\Utente\Desktop\weather-demo`.
- Al primo avvio Claude Code ha chiesto conferma di fiducia sulla cartella (controllo di sicurezza: potrà leggere/scrivere/eseguire lì) → concesso.
- Modello cambiato da Haiku 4.5 a **Opus 4.7** via `/model` (scelta motivata: task complesso, serve il modello più capace).
- Controllo dei tool MCP ("Quali tool Tiger hai a disposizione?") → l'agente ha elencato i tool `tiger`, confermando che l'MCP era agganciato.

Fase C — La build agentica (dal prompt all'app)

Prompt dato all'agente: costruire una dashboard meteo real-time con globo 3D, backend TimescaleDB via MCP (hypertable, aggregati continui, seeding 150 città + forecast), frontend Next.js + Three.js con slider time-travel e pannello di dettaglio.

L'agente ha proceduto a step, chiedendo conferma sulle azioni che modificano qualcosa:

1. `tiger – List Database Services` — prima si è orientato (verifica servizi esistenti: nessuno).
2. `tiger – Create Database Service` — creazione del servizio, con parametri **scelti autonomamente**: - `name: "weather-globe-demo"` - `addons: ["time-series"]` → ha attivato da solo l'estensione TimescaleDB - `region: "eu-central-1"` → region europea (Francoforte), rilevante per la residenza dei dati - `wait: true`, `with_password: true` - Servizio creato, `service_id: sjac3x02e4`.
3. **Schema** (via `db_execute_query`): - Tabella `cities` (`city_id` SERIAL PRIMARY KEY, name, country, latitude, longitude, timezone, `UNIQUE(name, country)`). - Tabella `weather_readings` (`time` TIMESTAMPTZ, `city_id` con FK, city_name, lat/lon, temperature, humidity, wind_speed, condition, e flag `is_forecast` **BOOLEAN DEFAULT FALSE** per separare storico e previsioni nella stessa tabella). - `create_hypertable('weather_readings', 'time', chunk_time_interval => INTERVAL '1 day')` → la tabella diventa hypertable, partizionata per tempo in chunk giornalieri. *Cuore di TimescaleDB.* - Indici `idx_weather_city_time (city_id, time DESC)` e `idx_weather_time (time DESC)` per velocizzare le query dello slider.
4. **Aggregati continui** (continuous aggregates): - `weather_hourly` — `time_bucket('1 hour', time)`, con avg/min/max temperatura, avg umidità, avg vento, per città. - `weather_daily` — `time_bucket('1 day', time)`, stesse metriche. - Entrambi `WITH (timescaledb.continuous)` e creati `WITH NO DATA` (vuoti, da popolare dopo il seeding). - **Refresh policy** (`add_continuous_aggregate_policy`): l'orario si aggiorna da 14 giorni a 1 ora fa, ogni ora; il giornaliero da 30 giorni a 1 giorno fa, ogni giorno → gli aggregati si mantengono aggiornati **da soli**.
5. **Seeding dei dati:** - `cities` popolata con **città reali** (coordinate vere e fuso orario, es. `Asia/Seoul`, `Europe/Istanbul`), `INSERT … ON CONFLICT (name, country) DO NOTHING` (idempotente). - **36.632 letture orarie** generate con un **modello stagionale + ciclo diurno** (più caldo all'equatore a giugno boreale, freddo a sud), **7 giorni storici + 3 di forecast** a partire da **2026-06-07 12:00 UTC**.
6. **Frontend** (file scritti in locale): - `Globe.tsx` — globo Three.js (sfera Phong + wireframe + atmosfera Fresnel via shader + ~2000 stelle + halo del sole), `OrbitControls`, **DirectionalLight posizionata al punto subsolare** → terminatore giorno/notte realistico che si sposta con lo slider. Importato in `Dashboard` con **dynamic import** `ssr: false` (carica il 3D solo nel browser — scelta corretta per Three.js, fatta autonomamente). - `TimeSlider.tsx` — lo slider time-travel (props hourOffset, hoursPast, hoursFuture, timestamp). -

`CityDetailPanel.tsx` — pannello di dettaglio con grafici **Recharts** (banda min/max, linea storica, linea tratteggiata forecast, reference line sul cursore temporale). - **API routes:** `/api/cities`, `/api/snapshot?ts=…`, `/api/city/[id]`.

Fase D — Validazione e avvio

- `npm install` (lanciato in background mentre l'agente verificava la struttura) → **175 pacchetti, exit code 0** (successo).
- `npx tsc --noEmit` → type check TypeScript **pulito** (l'agente valida il proprio codice prima di consegnarlo).
- Connection string del database già scritta in `.env.local` dall'agente.
- Avvio:

```
cd C:\Users\Utente\Desktop\weather-demo
npm run dev
```

→ **Next.js 14.2.5**, `Local: http://localhost:3000`, `.env.local` caricato, **Ready in 1501ms**.

---

5. Stato finale (cosa si vede a schermo)

Dashboard funzionante su `http://localhost:3000`: - Intestazione **WEATHER GLOBE** — "*Tiger Cloud · TimescaleDB · 152 cities*" (dati letti dal database reale). - Pannello *HOTTEST NOW* **Mogadishu 29.6 °C**, *COLDEST NOW* **Ushuaia −14.9 °C**. - Timestamp *HISTORICAL* **2026-06-07 12:00 UTC**. - Globo 3D con marker città colorati per temperatura (azzurro→giallo→arancio) e terminatore giorno/notte. - Slider **TIME TRAVEL** da −7d a +3d (storico a sinistra di "now", forecast a destra). - Interazioni: trascinare per ruotare, rotella per zoom, click su marker per il dettaglio.

---

6. Momenti didattici osservati durante la build

Punti concreti emersi dal vivo, utili da far notare in aula (l'agente che *ragiona*, non solo esegue):

- Si è **orientato prima di agire** (ha listato i servizi esistenti prima di crearne uno).
- Ha **scelto da solo** l'addon `time-series` e una **region europea**, senza indicazione esplicita.
- Ha usato `ON CONFLICT DO NOTHING` → inserimenti **idempotenti**.
- Ha generato dati **realistici** (modello stagionale/diurno), non numeri casuali.
- Ha riconosciuto il problema del rendering server-side di Three.js e l'ha prevenuto con `dynamic import ssr:false`.
- Ha lanciato l'install **in background** lavorando in parallelo (non è rimasto fermo ad aspettare).
- Ha **validato il proprio lavoro** con un type check prima di consegnare.

E i guardrail visti in azione: la conferma di fiducia sulla cartella e i permessi richiesti prima di ogni azione che crea/modifica — l'utente resta nel loop. (Analogo, concettualmente, alla cautela del `curl … | sh`: dare all'AI il potere di *agire* richiede consenso esplicito.)

---

7. Nota onesta: i dati sono sintetici

Le letture **non sono dati meteo reali**: sono generate da un modello costruito dall'agente. Per il messaggio della demo è un **punto a favore** ("guardate come l'AI ha capito da sola che i dati dovevano essere verosimili"), non un limite.

Per estenderla a dati reali (opzionale, task aggiuntivo): collegare una fonte come Open-Meteo (gratuita, senza chiave), far scrivere all'agente uno script che la interroga per le 150 città e popola la stessa `weather_readings`. Lo schema TimescaleDB e il frontend **non cambierebbero** — solo la sorgente dei dati. Da valutare però che questo sposta il focus verso l'integrazione di API, tema più ingegneristico e meno adatto a un pubblico che vuole capire il *concetto* di agente.

Coordinate del progetto (riferimento rapido)

- **Cartella locale:** `C:\Users\Utente\Desktop\weather-demo`
- **Servizio Tiger Cloud:** `weather-globe-demo` · id `sjac3x02e4` · region `eu-central-1`
- **Progetto Tiger Cloud:** `pb9zqxrnzp`
- **DB:** TimescaleDB — tabelle `cities`, `weather_readings` (hypertable); aggregati `weather_hourly`, `weather_daily`
- **Dati:** 152 città · 36.632 letture · 7gg storici + 3gg forecast · riferimento 2026-06-07 12:00 UTC
- **Frontend:** Next.js 14.2.5 + Three.js + Recharts + Tailwind · avvio `npm run dev` → `http://localhost:3000`
- **Config DB:** `.env.local` (connection string già presente)

---

8. Glossario dei termini tecnici

Tutti i termini incontrati nella sessione, raggruppati per area. Per il taglio "capire l'AI", i concetti centrali sono MCP, TimescaleDB /

hypertable e Tiger Cloud; gli altri sono di riferimento.

Riga di comando e shell

`cmd` **(Prompt dei comandi)** — La shell tradizionale di Windows. Comprende i comandi DOS/Windows ma non quelli di PowerShell: per questo `irm` / `iex` lanciati lì danno "non riconosciuto come comando".

`curl` — Strumento da riga di comando per trasferire dati tramite URL (scaricare o inviare). Tipicamente usato per richieste HTTP da terminale, ad esempio scaricare uno script o interrogare un'API. Su Windows 10+ è incluso nativamente.

Execution policy — Un'impostazione di sicurezza di PowerShell che regola se e quali script possono essere eseguiti. Se troppo restrittiva blocca l'installazione; si può allentare solo per la sessione corrente con `Set-ExecutionPolicy -Scope Process`.

`iex` **(Invoke-Expression)** — Comando PowerShell che esegue come codice il testo che riceve. Abbinato a `irm` (`irm … | iex`) realizza lo schema "scarica uno script ed eseguilo".

`irm` **(Invoke-RestMethod)** — Comando PowerShell che scarica contenuto da un URL; è l'equivalente Windows di `curl` per questo scopo nello schema di installazione.

npm — Il gestore di pacchetti di Node.js: scarica e installa le librerie di cui un progetto JavaScript ha bisogno (qui React, Three.js, ecc.). `npm install` installa le dipendenze, `npm run dev` avvia il server di sviluppo.

npx — Comando affiancato a npm che esegue un pacchetto senza installarlo stabilmente. Nella demo è stato usato per `npx tsc` (il controllo dei tipi TypeScript).

PATH — Una variabile di sistema che elenca le cartelle in cui il sistema cerca i programmi eseguibili. Quando un installer aggiunge un comando al PATH, quel comando diventa richiamabile da qualsiasi cartella; una finestra di terminale già aperta va però riaperta per leggere il PATH aggiornato.

Pipe (`|`) — Il simbolo che collega due comandi facendo sì che l'output del primo diventi l'input del secondo. In `curl … | sh` lo script scaricato non viene mostrato ma passato direttamente alla shell che lo esegue.

PowerShell — La shell moderna di Windows, più potente di `cmd`, con un proprio linguaggio. È l'ambiente corretto per i comandi `irm` / `iex` usati nell'installazione su Windows.

`sh` — La shell storica di Unix (Bourne shell). Su Linux e macOS è un interprete di comandi predefinito; su Windows non esiste in modo nativo (si ottiene con Git Bash, WSL o simili). Negli script "scarica ed esegui" riceve ed esegue il codice scaricato.

Shell — Il programma che interpreta i comandi testuali che digiti e li fa eseguire al sistema operativo. È l'intermediario tra te e la macchina quando lavori da terminale anziché con il mouse.

Strumenti e prodotti

Claude Code — L'assistente AI agentico di Anthropic che opera da terminale: legge e scrive file, esegue comandi e — collegato via MCP — agisce su servizi esterni. È l'agente che ha costruito l'intera applicazione.

MCP (Model Context Protocol) — Lo standard che fa da ponte tra un'AI e strumenti o servizi esterni reali. Senza, l'AI produce solo testo; con un server MCP collegato può azionare strumenti veri — qui creare il database ed eseguire query SQL su Tiger Cloud. È il meccanismo che rende l'AI capace di *agire*.

Modello / effort (Opus 4.7, xhigh) — Il "modello" è la versione di AI che l'agente usa; Opus è la più capace per compiti complessi. L'"effort" regola quanto a fondo il modello ragiona prima di rispondere: xhigh è il livello massimo, adatto a un task lungo e articolato come questo.

Region (regione cloud) — La collocazione geografica dei server su cui gira un servizio cloud. L'agente ha scelto `eu-central-1` (Francoforte): rilevante per la residenza dei dati, ad esempio in presenza di vincoli europei.

Tiger CLI — Lo strumento da riga di comando (`tiger`) per gestire le risorse di Tiger Cloud: creare servizi, eseguire query, ecc. Include al suo interno il server MCP.

Tiger Cloud — Il servizio gestito di Tiger Data: PostgreSQL + TimescaleDB già installati e mantenuti nel loro cloud, così non devi gestire tu un server. È il database che l'agente ha creato a distanza nella demo.

Tiger Data — L'azienda che sviluppa TimescaleDB e offre Tiger Cloud (è il nuovo nome di Timescale).

TimescaleDB — Un'estensione di PostgreSQL specializzata per le serie temporali, cioè dati che cambiano nel tempo (sensori, prezzi, log, rilevazioni meteo). Aggiunge a Postgres strumenti per gestire grandi volumi di dati temporali mantenendo le query veloci, pur restando Postgres a tutti gli effetti (SQL, transazioni ed ecosistema noti).

Database e SQL

Chiave esterna (foreign key / REFERENCES) — Un riferimento che lega una riga di una tabella a una riga di un'altra, garantendo la coerenza. Nella demo `weather_readings.city_id` punta a una città esistente in `cities`.

Chiave primaria / SERIAL — La chiave primaria è la colonna che identifica univocamente ogni riga di una tabella. `SERIAL` è un tipo che assegna automaticamente un numero progressivo a ogni nuova riga (usato per `city_id`).

Chunk — Ciascuno dei pezzi in cui una hypertable suddivide i dati. Nella demo l'intervallo è di un giorno (`chunk_time_interval => INTERVAL '1 day'`): ogni chunk contiene le letture di una giornata.

Connection string — La stringa che contiene le informazioni per collegarsi a un database (indirizzo, credenziali, nome). Nella demo è salvata nel file `.env.local` e usata dal frontend per leggere da TimescaleDB.

Continuous aggregate (aggregato continuo) — Una vista che contiene risultati riassuntivi precalcolati (es. medie orarie o giornaliere) e che TimescaleDB mantiene aggiornata automaticamente. Permette ai grafici di leggere valori già pronti invece di ricalcolarli ogni volta sui dati grezzi.

Funzioni di aggregazione (AVG, MIN, MAX, bool_or) — Funzioni SQL che riassumono più righe in un valore: media, minimo, massimo, e `bool_or` (vero se almeno una riga è vera). Usate negli aggregati per calcolare temperatura media/min/max per intervallo.

Hypertable — La struttura distintiva di TimescaleDB: una tabella che si usa come una normale, ma che dietro le quinte viene spezzata automaticamente in più "chunk" per intervallo di tempo. Così una query su un periodo legge solo i chunk pertinenti invece dell'intera tabella, restando veloce anche con molti dati.

Idempotenza — La proprietà di un'operazione che, ripetuta più volte, produce sempre lo stesso risultato senza effetti collaterali. Il seeding con `ON CONFLICT DO NOTHING` è idempotente: rilanciarlo non duplica i dati.

Indice (index) — Una struttura ausiliaria che velocizza la ricerca su determinate colonne, come l'indice analitico di un libro. Nella demo gli indici su `city_id` e `time` accelerano le query dello slider.

Materialized view (vista materializzata) — Una vista i cui risultati sono salvati su disco anziché ricalcolati a ogni lettura. I continuous aggregates sono un tipo speciale di vista materializzata con aggiornamento automatico.

`ON CONFLICT DO NOTHING` — Una clausola SQL che, in caso di conflitto con un vincolo (es. una città già presente), ignora l'inserimento invece di dare errore. Rende il seeding ripetibile senza creare duplicati.

PostgreSQL (Postgres) — Uno dei database relazionali open source più diffusi al mondo. Un database relazionale conserva i dati in tabelle con righe e colonne, ed è la base su cui poggia tutto il backend della demo.

Query — Una singola istruzione SQL rivolta al database, ad esempio per leggere o inserire dati. Ogni movimento dello slider della demo genera una query verso TimescaleDB.

Refresh policy — La regola che dice a TimescaleDB ogni quanto e su quale finestra aggiornare un aggregato continuo (`add_continuous_aggregate_policy`). Nella demo gli aggregati si aggiornano da soli a intervalli definiti.

Seeding — Il caricamento iniziale di dati in un database vuoto per renderlo utilizzabile. Qui ha riguardato le 152 città e le 36.632 letture meteo.

SQL — Il linguaggio standard per interrogare e manipolare i dati di un database relazionale (creare tabelle, inserire righe, fare ricerche). Tutte le operazioni sul database, nella demo, sono comandi SQL eseguiti dall'agente.

`time_bucket` — Una funzione di TimescaleDB che raggruppa i dati in intervalli di tempo regolari (ogni ora, ogni giorno). Sostituisce il raggruppamento manuale per fasce temporali che in Postgres puro sarebbe più macchinoso.

TIMESTAMPTZ — Il tipo di dato PostgreSQL per una data e ora con fuso orario ("timestamp with time zone"). È la colonna `time` su cui si basa la hypertable.

Vincolo UNIQUE — Una regola che impedisce valori duplicati su una colonna o combinazione di colonne. Qui `UNIQUE (name, country)` evita due città identiche.

Web e frontend

API route — In Next.js, un punto di accesso lato server che il frontend chiama per ottenere dati. Le route `/api/cities`, `/api/snapshot`, `/api/city/[id]` interrogano il database e restituiscono i dati al globo.

DirectionalLight / punto subsolare — La `DirectionalLight` è una luce direzionale di Three.js (come il sole). Posizionandola al "punto subsolare" (il punto della Terra dove il Sole è allo zenit in un dato istante) si ottiene il terminatore giorno/notte realistico che si muove con lo slider.

`.env.local` — Un file che contiene le variabili d'ambiente locali, tra cui dati riservati come la connection string del database. Resta sulla tua macchina e non finisce nel codice condiviso.

Exit code — Il codice numerico restituito da un comando al termine: 0 significa successo, diverso da 0 indica un errore. L'`npm install` concluso con "exit code 0" ha confermato il buon esito.

LTS (Long Term Support) — La versione di un software con supporto a lungo termine, la più stabile e consigliata per l'uso reale. La Node.js installata (v22) è una LTS.

Next.js — Un framework per costruire applicazioni web moderne basate su React, con routing, rendering e API integrate. È lo scheletro del frontend della demo.

Node.js — Un ambiente che permette di eseguire codice JavaScript fuori dal browser. È il "motore" su cui girano sia Claude Code sia il server di sviluppo del sito.

OrbitControls — Un modulo di Three.js che permette di ruotare e zoomare la scena 3D col mouse. Nella demo abilita il "trascina per ruotare il globo".

React — La libreria JavaScript per costruire interfacce a "componenti" riutilizzabili. I file `.tsx` (TimeSlider, CityDetailPanel, Globe) sono componenti React.

Recharts — Una libreria per disegnare grafici in React. Genera le linee di temperatura (storico, forecast, banda min/max) del pannello di dettaglio città.

Server di sviluppo (`npm run dev`), localhost e porta — `npm run dev` avvia un server locale per provare l'app in sviluppo. "localhost" è la tua stessa macchina e ":3000" è la porta su cui il server ascolta: l'indirizzo `http://localhost:3000` apre la dashboard nel browser.

Shader / effetto Fresnel — Uno "shader" è un piccolo programma che calcola l'aspetto dei pixel di una grafica 3D. L'effetto "Fresnel"

simula il bagliore atmosferico ai bordi del globo, dove la luce ne sfiora la superficie.

SSR (Server-Side Rendering) e dynamic import — L'SSR è la generazione delle pagine sul server prima di inviarle al browser. Three.js ha bisogno del browser, quindi l'agente ha caricato il globo con un "dynamic import" e `ssr: false`, cioè solo lato client, evitando errori.

Tailwind (CSS) — Un framework di stili che permette di impaginare e decorare l'interfaccia con classi predefinite. Dà alla dashboard il suo tema scuro.

Three.js — La libreria che disegna grafica 3D nel browser (sfere, luci, materiali). È ciò che rende possibile il globo della demo.

`.tsx` / `.ts` — Le estensioni dei file TypeScript: `.tsx` per i file che contengono anche markup di interfaccia (componenti React), `.ts` per il codice puro.

Type check (`tsc --noEmit`) — Il controllo che verifica la correttezza dei tipi nel codice TypeScript senza produrre file (`--noEmit` = "non generare output"). L'agente l'ha usato per validare il proprio codice prima di consegnarlo.

TypeScript — Un'estensione di JavaScript che aggiunge i "tipi", cioè controlli sulla natura dei dati, riducendo gli errori. I file `.ts` / `.tsx` del progetto sono TypeScript.

`"use client"` — Una direttiva di Next.js che marca un componente come eseguito nel browser (lato client) anziché sul server. Necessaria per i componenti interattivi e per la grafica 3D.

Altri concetti

Binario (binary) — Un file di programma già compilato e pronto all'esecuzione, come il comando `tiger` scaricato dall'installer.

Checksum — Un codice di verifica calcolato su un file per controllare che sia integro e non alterato. L'installer della CLI ha verificato il checksum del binario scaricato.

Comando in background — Un comando lasciato in esecuzione mentre se ne fanno altri, senza bloccare il terminale. L'agente ha lanciato `npm install` in background continuando a lavorare nel frattempo.

conda — Un gestore di ambienti e pacchetti molto usato in ambito Python e data science. Il prompt `(base)` indicava l'ambiente conda attivo; è stato disattivato perché non serviva agli strumenti Node-based.

Telemetria — La raccolta automatica e anonima di dati d'uso da parte di un software per migliorarlo. Next.js ne raccoglie un po' all'avvio; è solo una nota informativa, disattivabile.